

DEVIXO SOLUTIONS

DATA SCIENCE INTERNSHIP

ARAMISH ADNAN

TASK 01

OBJECTIVE

Perform exploratory data analysis on e-commerce transactions using Python. The goal is to clean the dataset, uncover purchasing patterns, analyze customer demographics, and deliver actionable recommendations to boost store sales and lower product return rates.

PROBLEM STATEMENT

The store struggles with uneven category performance, profit loss from product returns, and ineffective promotions. Flat discounts fail to grow basket sizes, while high refund rates in Fashion and Electronics cut into profit margins. Without clear data insights, stock allocation and promotional spending remain inefficient.

DATA DESCRIPTION

The analysis uses the Kaggle E-commerce Sales Transactions dataset. It contains 34,500 raw order records across 17 features, combining sales figures, customer demographics, and shipping details.

- **Dataset Source:** Kaggle (E-commerce Sales Transactions Dataset)
<https://www.kaggle.com/datasets/miadul/e-commerce-sales-transactions-dataset>
- **Dataset Size:** 34,500 rows × 17 columns

Feature Breakdown

- **Order Identifiers:** order_id, customer_id, product_id
- **Financial Metrics:** price, discount, total_amount, profit_margin, shipping_cost
- **Product & Logistics:** category, quantity, order_date, delivery_time_days, returned
- **Customer Demographics:** customer_age, customer_gender, region, payment_method

TECHNOLOGIES USED

No.	Tech
1	Google Colab
2	Pandas
3	Numpy
4	Matplotlib
5	Seaborn
6	Github

DATA IMPORT & UNDERSTANDING

```
import pandas as pd
```

```
df= pd.read_csv('E-commerce.csv')  
df.head()
```

```
...      order_id  customer_id  product_id  category  price  discount  quantity  payment_method  order_date  deli
```

0	O100000	C17270	P234890	Home	164.08	0.15	1	Credit Card	2023-12-23	
1	O100001	C17603	P228204	Grocery	24.73	0.00	1	Credit Card	2025-04-03	
2	O100002	C10860	P213892	Electronics	175.58	0.05	1	Credit Card	2024-10-08	
3	O100003	C15390	P208689	Electronics	63.67	0.00	1	UPI	2024-09-14	
4	O100004	C15226	P228063	Home	16.33	0.15	1	COD	2024-12-21	

```
df.tail()
```

```
...      order_id  customer_id  product_id  category  price  discount  quantity  payment_method  order_date  de
```

34495	O134495	C13553	P231412	Fashion	10.44	0.20	1	Debit Card	2025-08-13	
34496	O134496	C11349	P218266	Beauty	4.79	0.10	5	Debit Card	2023-12-04	
34497	O134497	C12986	P204345	Electronics	323.64	0.00	5	Credit Card	2025-02-01	
34498	O134498	C17196	P217644	Home	23.09	0.05	1	Credit Card	2024-03-27	
34499	O134499	C13330	P207047	Home	76.84	0.00	1	Debit Card	2025-09-11	

```
print(f"Dataset Shape: {df.shape[0]} rows, {df.shape[1]} columns")
```

```
Dataset Shape: 34500 rows, 17 columns
```

```
df.columns
```

```
... Index(['order_id', 'customer_id', 'product_id', 'category', 'price',  
         'discount', 'quantity', 'payment_method', 'order_date',  
         'delivery_time_days', 'region', 'returned', 'total_amount',  
         'shipping_cost', 'profit_margin', 'customer_age', 'customer_gender'],  
         dtype='object')
```



df.info()

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 34500 entries, 0 to 34499
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   order_id              34500 non-null  object
1   customer_id           34500 non-null  object
2   product_id            34500 non-null  object
3   category              34500 non-null  object
4   price                 34500 non-null  float64
5   discount              34500 non-null  float64
6   quantity              34500 non-null  int64
7   payment_method        34500 non-null  object
8   order_date            34500 non-null  object
9   delivery_time_days    34500 non-null  int64
10  region                34500 non-null  object
11  returned              34500 non-null  object
12  total_amount          34500 non-null  float64
13  shipping_cost         34500 non-null  float64
14  profit_margin         34500 non-null  float64
15  customer_age          34500 non-null  int64
16  customer_gender       34500 non-null  object
dtypes: float64(5), int64(3), object(9)
memory usage: 4.5+ MB
```



```
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_cols = df.select_dtypes(include=['object', 'category', 'bool']).columns.tolist()
print(numerical_cols)
print(categorical_cols)

['price', 'discount', 'quantity', 'delivery_time_days', 'total_amount', 'shipping_cost', 'profit_margin', 'customer_age']
['order_id', 'customer_id', 'product_id', 'category', 'payment_method', 'order_date', 'region', 'returned', 'customer_gender']
```



df.describe()

```
...

```

	price	discount	quantity	delivery_time_days	total_amount	shipping_cost	profit_margin	customer_age
count	34500.000000	34500.000000	34500.000000	34500.000000	34500.000000	34500.000000	34500.000000	34500.000000
mean	119.391632	0.049291	1.490725	4.814203	170.008494	6.152120	28.116505	43.474377
std	195.620477	0.069894	0.932270	1.242141	357.503014	2.389539	53.352947	14.980682
min	1.010000	0.000000	1.000000	3.000000	0.820000	0.000000	-6.200000	18.000000
25%	16.690000	0.000000	1.000000	4.000000	19.710000	4.420000	1.500000	31.000000
50%	45.660000	0.000000	1.000000	5.000000	56.820000	6.090000	10.550000	43.000000
75%	130.950000	0.100000	2.000000	6.000000	168.530000	7.830000	33.132500	56.000000
max	2930.470000	0.300000	5.000000	13.000000	12931.800000	15.650000	1536.170000	69.000000



```
df.describe(include=['object', 'bool']).T
```

...

	count	unique	top	freq
order_id	34500	34500	O134499	1
customer_id	34500	7903	C16704	13
product_id	34500	24912	P200645	7
category	34500	7	Fashion	6254
payment_method	34500	6	Credit Card	12170
order_date	34500	731	2024-12-04	70
region	34500	5	South	7584
returned	34500	2	No	32597
customer_gender	34500	3	Female	16818

DATA CLEANING

```
import numpy as np

df.isnull().sum()

0
order_id      0
customer_id   0
product_id    0
category      0
price         0
discount      0
quantity      0
payment_method 0
order_date    0
delivery_time_days 0
region        0
returned      0
total_amount  0
shipping_cost 0
profit_margin 0
customer_age  0
customer_gender 0
```

```
df.duplicated().sum()

np.int64(0)

df['order_date'] = pd.to_datetime(df['order_date'], errors='coerce')

cat_cols = ['category', 'payment_method', 'region']
for col in cat_cols:
    if col in df.columns and df[col].dtype == 'object':
        df[col] = df[col].str.strip().str.title()

if 'price' in df.columns:
    df = df[df['price'] > 0]
if 'quantity' in df.columns:
    df = df[df['quantity'] > 0]
if 'discount' in df.columns:
    df = df[(df['discount'] >= 0) & (df['discount'] <= 100)]
```

FEATURE ANALYSIS

```
if 'total_sales' not in df.columns:
    df['total_sales'] = df['price'] * df['quantity'] * (1 - (df['discount'] / 100))

num_features = ['price', 'quantity', 'discount', 'delivery_time_days', 'total_sales']
num_summary = []

for col in num_features:
    if col in df.columns:
        q1 = df[col].quantile(0.25)
        q3 = df[col].quantile(0.75)
        iqr = q3 - q1
        lower_bound = q1 - 1.5 * iqr
        upper_bound = q3 + 1.5 * iqr
        outliers_count = df[(df[col] < lower_bound) | (df[col] > upper_bound)].shape[0]

        num_summary.append({
            'Feature': col,
            'Min': round(df[col].min(), 2),
            'Max': round(df[col].max(), 2),
            'Mean': round(df[col].mean(), 2),
            'Median': round(df[col].median(), 2),
            'Std Dev': round(df[col].std(), 2),
            'Skewness': round(df[col].skew(), 2),
            'Outliers Count': outliers_count,
            'Outlier %': round((outliers_count / len(df)) * 100, 2)
        })

num_df = pd.DataFrame(num_summary)
display(num_df)
```

	Feature	Min	Max	Mean	Median	Std Dev	Skewness	Outliers Count	Outlier %
0	price	1.01	2930.47	119.39	45.66	195.62	3.70	3632	10.53
1	quantity	1.00	5.00	1.49	1.00	0.93	2.05	2070	6.00
2	discount	0.00	0.30	0.05	0.00	0.07	1.54	704	2.04
3	delivery_time_days	3.00	13.00	4.81	5.00	1.24	0.56	27	0.08
4	total_sales	1.01	12931.80	178.86	59.73	374.65	7.89	3770	10.93

```
cat_features = ['category', 'payment_method', 'region', 'returned']
cat_summary = []
```

```
for col in cat_features:
    if col in df.columns:
        top_cat = df[col].mode()[0]
        top_freq = df[col].value_counts().iloc[0]
        top_pct = (top_freq / len(df)) * 100

        cat_summary.append({
            'Feature': col,
            'Unique Categories': df[col].nunique(),
            'Top Category': top_cat,
            'Top Category Freq': top_freq,
            'Top Category %': round(top_pct, 2)
        })
```

```
cat_df = pd.DataFrame(cat_summary)
display(cat_df)
```

```
...
```

	Feature	Unique Categories	Top Category	Top Category Freq	Top Category %
0	category	7	Fashion	6254	18.13
1	payment_method	6	Credit Card	12170	35.28
2	region	5	South	7584	21.98
3	returned	2	No	32597	94.48

```
for col in ['category', 'payment_method', 'region']:
    if col in df.columns:
        print(f"\nDistribution for '{col}':")
        print(df[col].value_counts(normalize=True).map(lambda n: f"{n:.2%}"))
```

...

Distribution for 'category':

category

Fashion 18.13%

Electronics 17.91%

Home 15.90%

Toys 12.31%

Sports 12.09%

Beauty 11.89%

Grocery 11.76%

Name: proportion, dtype: object

Distribution for 'payment_method':

payment_method

Credit Card 35.28%

Debit Card 24.65%

Cod 12.06%

UPI 12.05%

Paypal 9.98%

Wallet 5.99%

Name: proportion, dtype: object

Distribution for 'region':

region

South 21.98%

North 21.95%

East 20.01%

West 19.73%

Central 16.32%

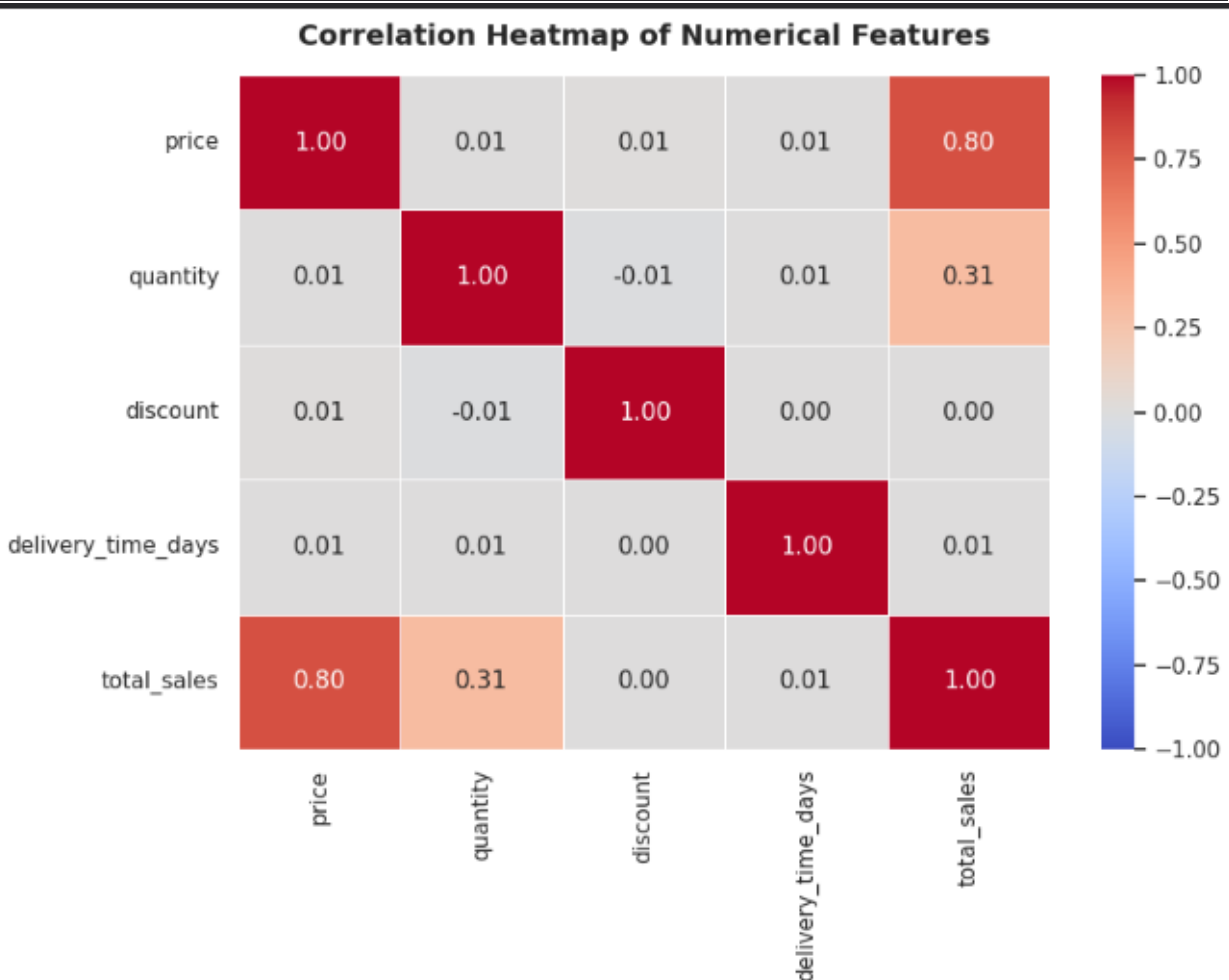
Name: proportion, dtype: object

CORRELATION ANALYSIS

```
import matplotlib.pyplot as plt
import seaborn as sns

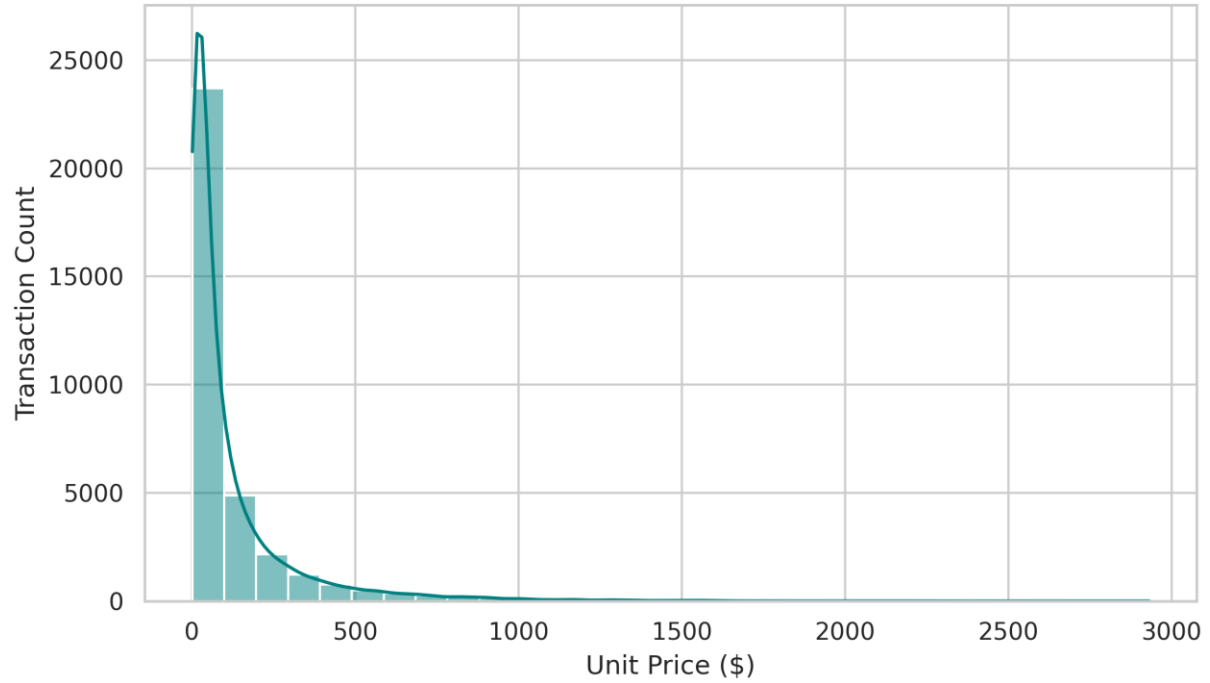
num_cols = ['price', 'quantity', 'discount', 'delivery_time_days', 'total_sales']
corr_matrix = df[num_cols].corr()

plt.figure(figsize=(9, 7))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5, vmin=-1, vmax=1)
plt.title('Correlation Heatmap of Numerical Features', fontsize=14, fontweight='bold', pad=15)
plt.tight_layout()
plt.show()
```

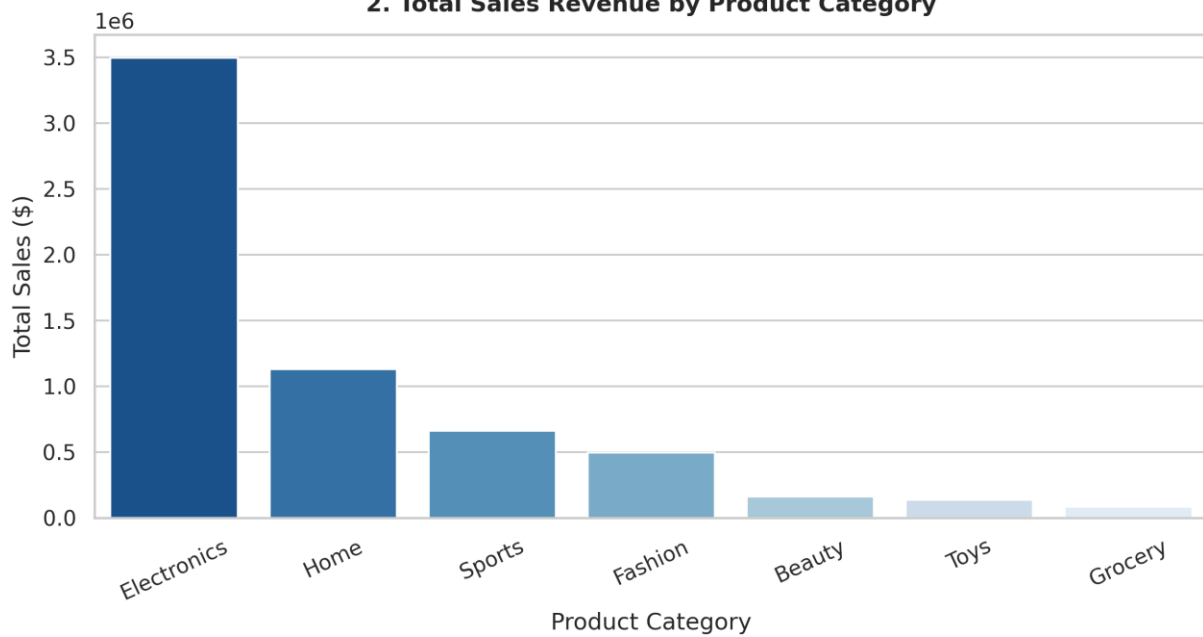


DATA VISUALIZATION

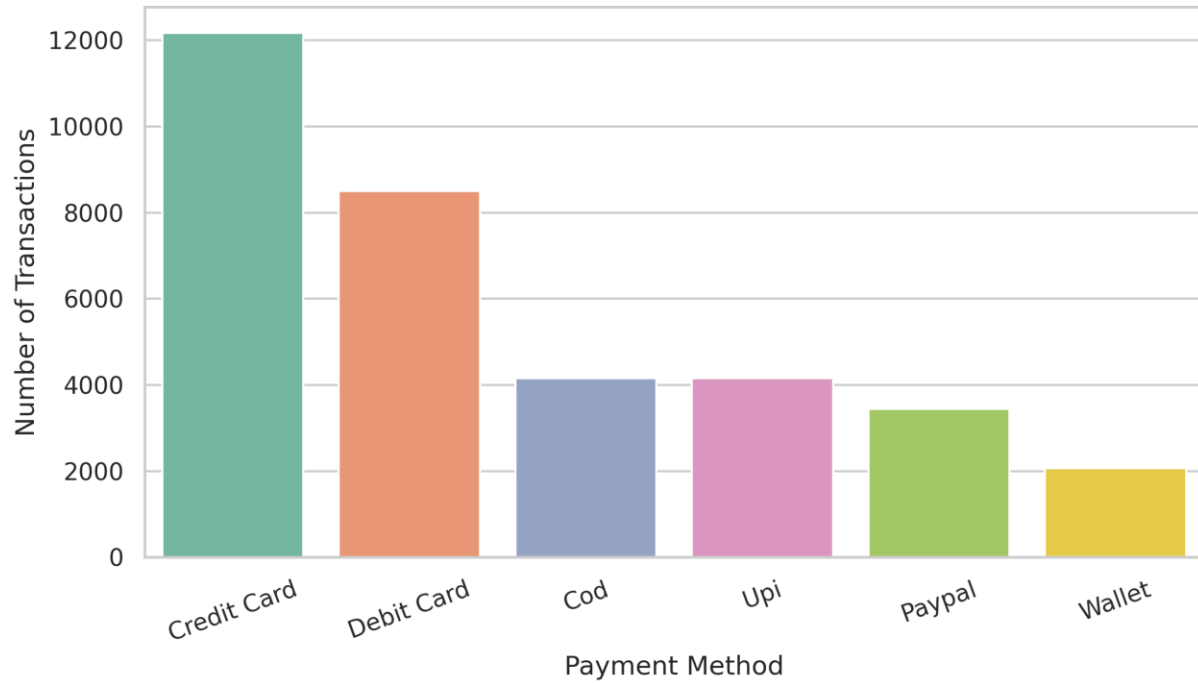
1. Unit Price Distribution



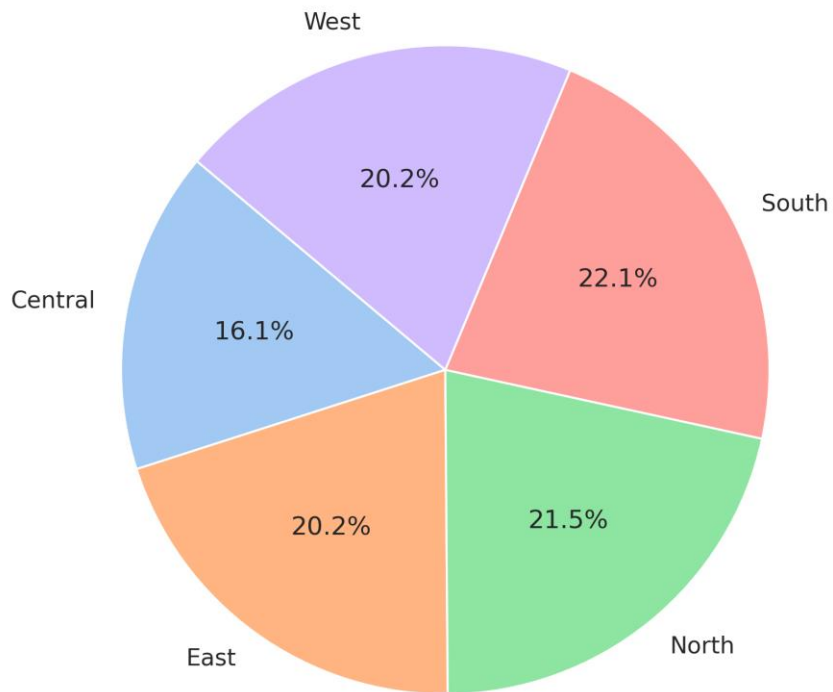
2. Total Sales Revenue by Product Category



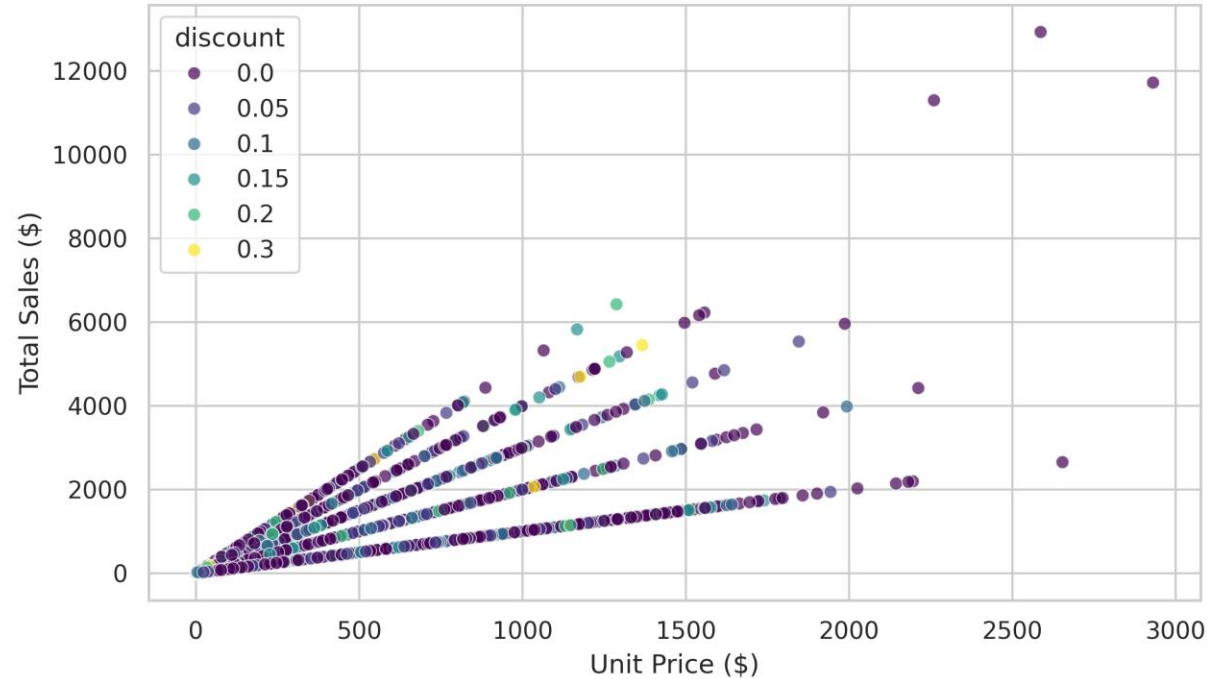
3. Transaction Count by Payment Method



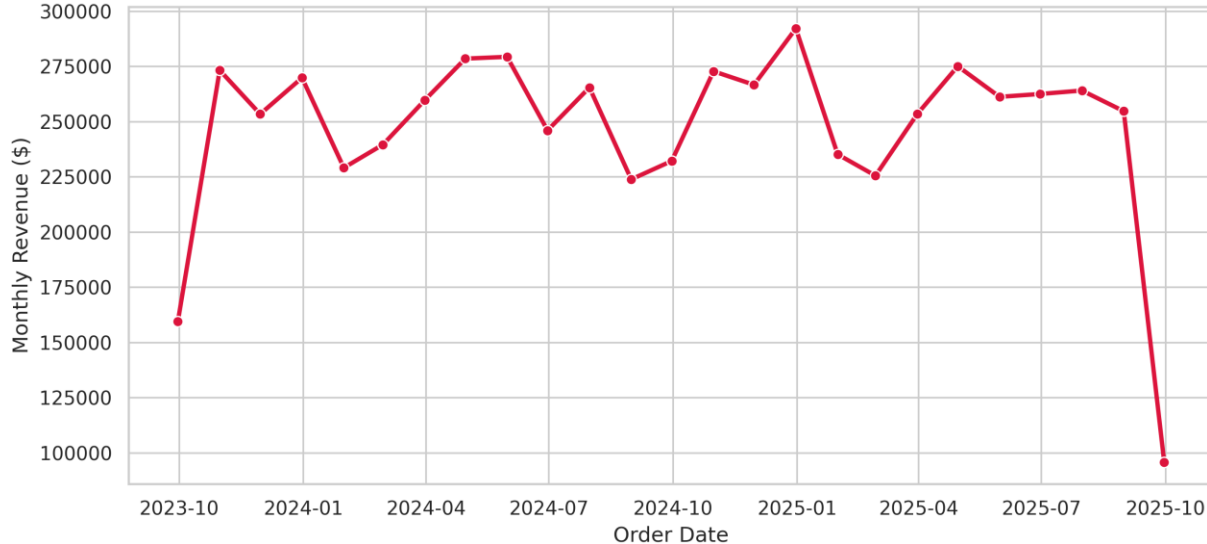
4. Regional Sales Revenue Share



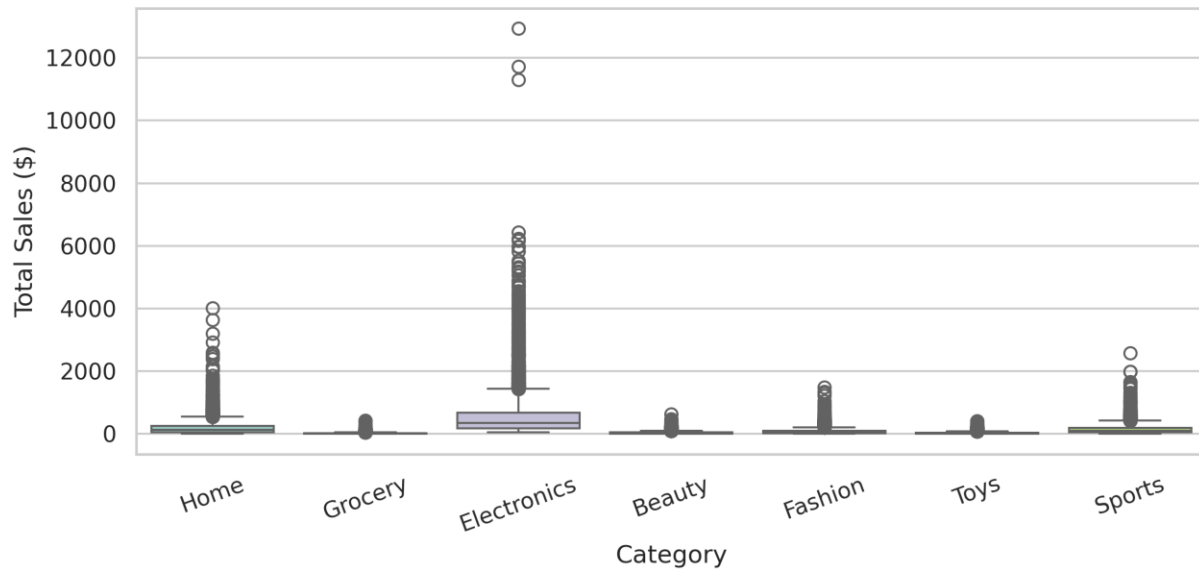
5. Unit Price vs. Total Sales Value (by Discount %)



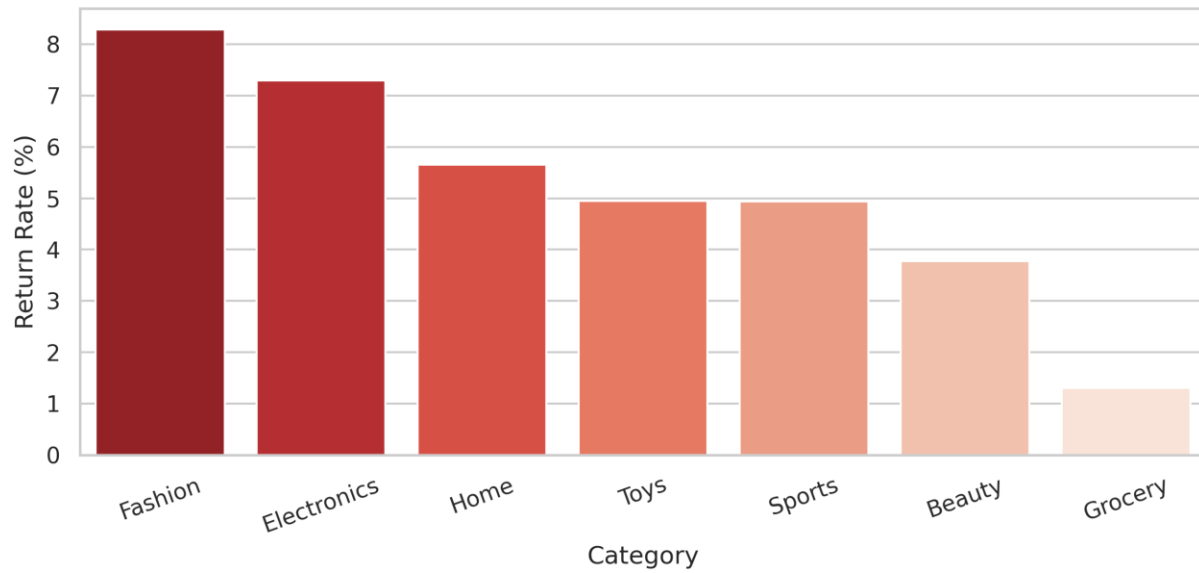
6. Monthly Revenue Trend Over Time



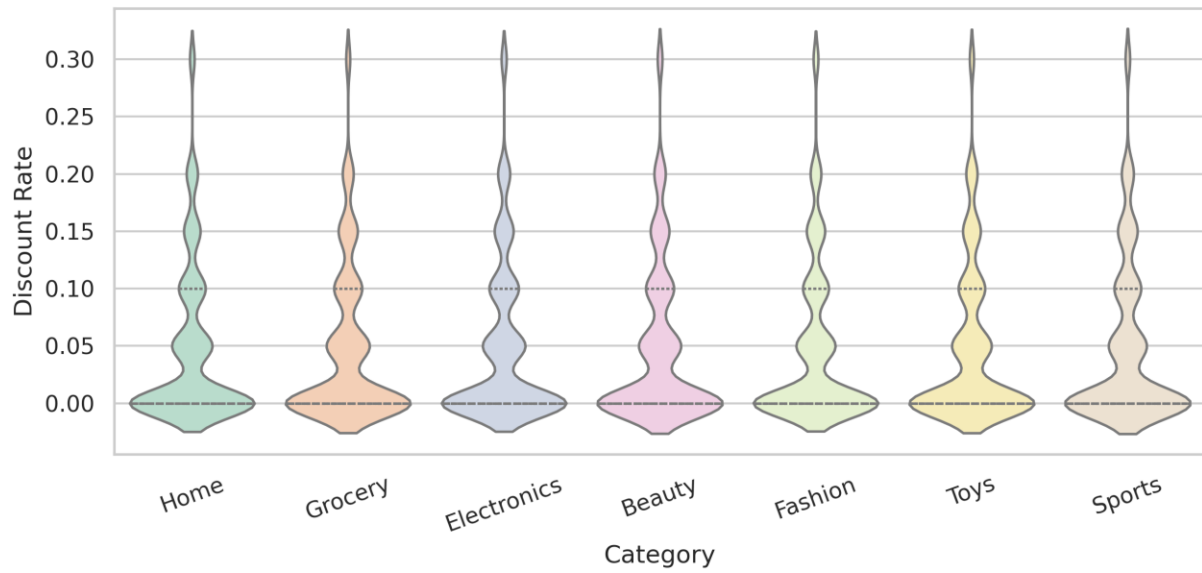
7. Transaction Value Outliers & Variance by Category



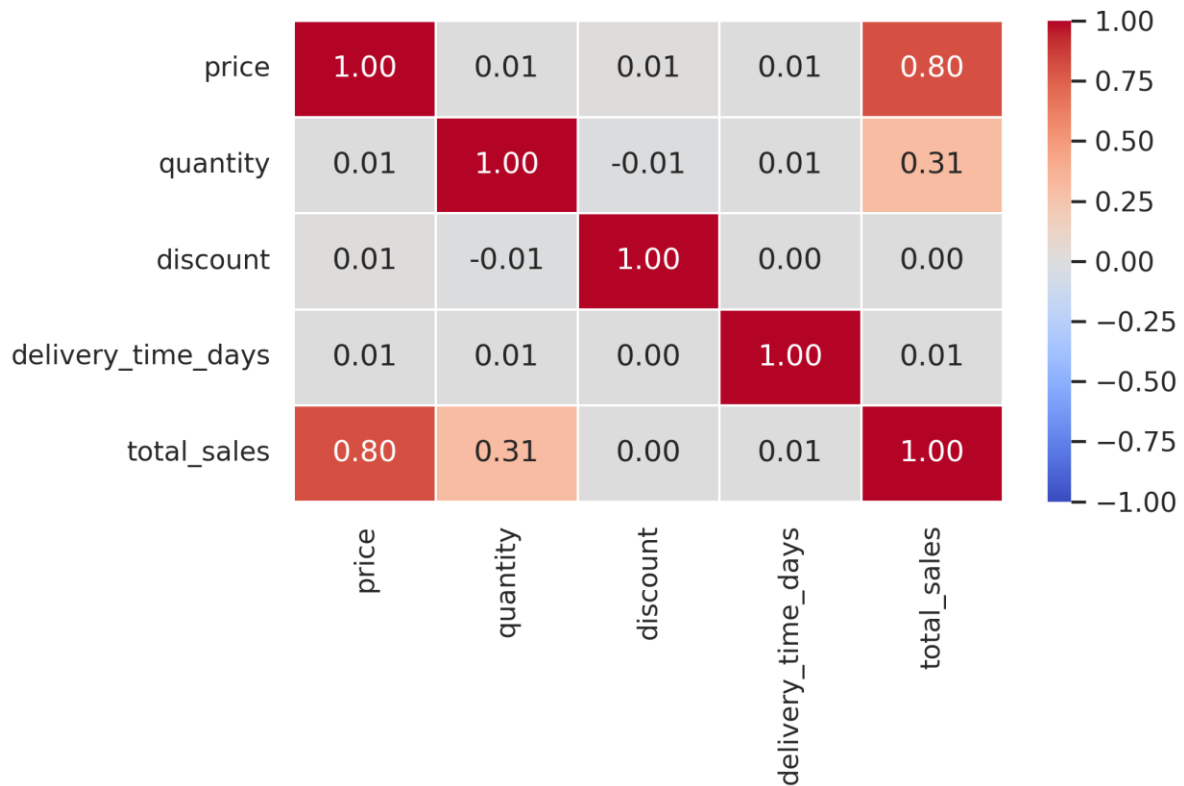
8. Product Return Rate (%) by Category



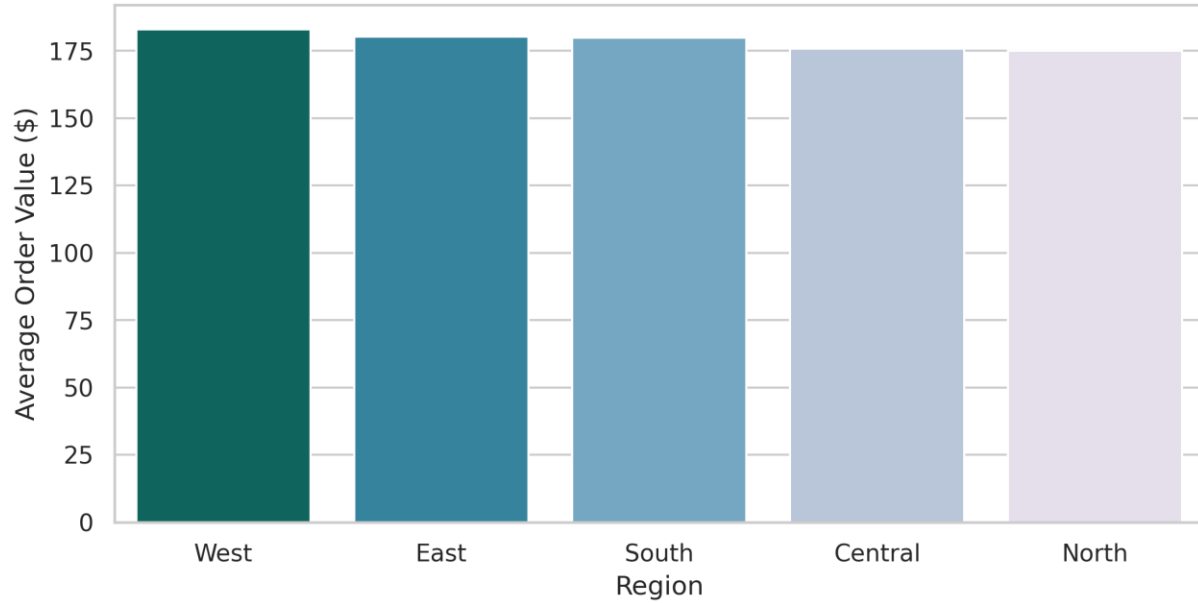
9. Promotional Discount Distribution Across Categories



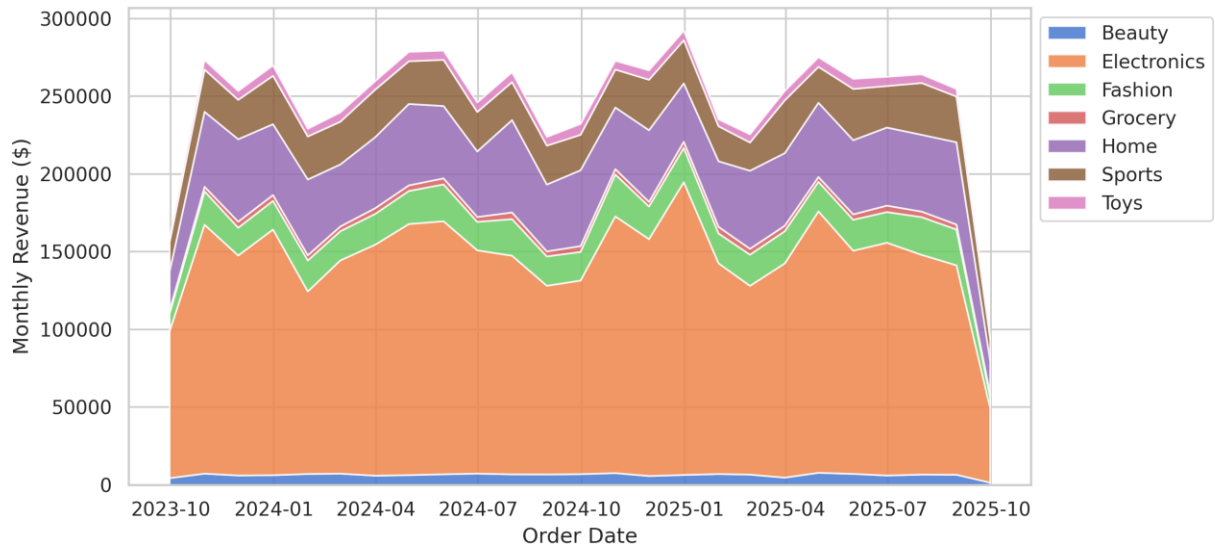
10. Numerical Feature Correlation Matrix

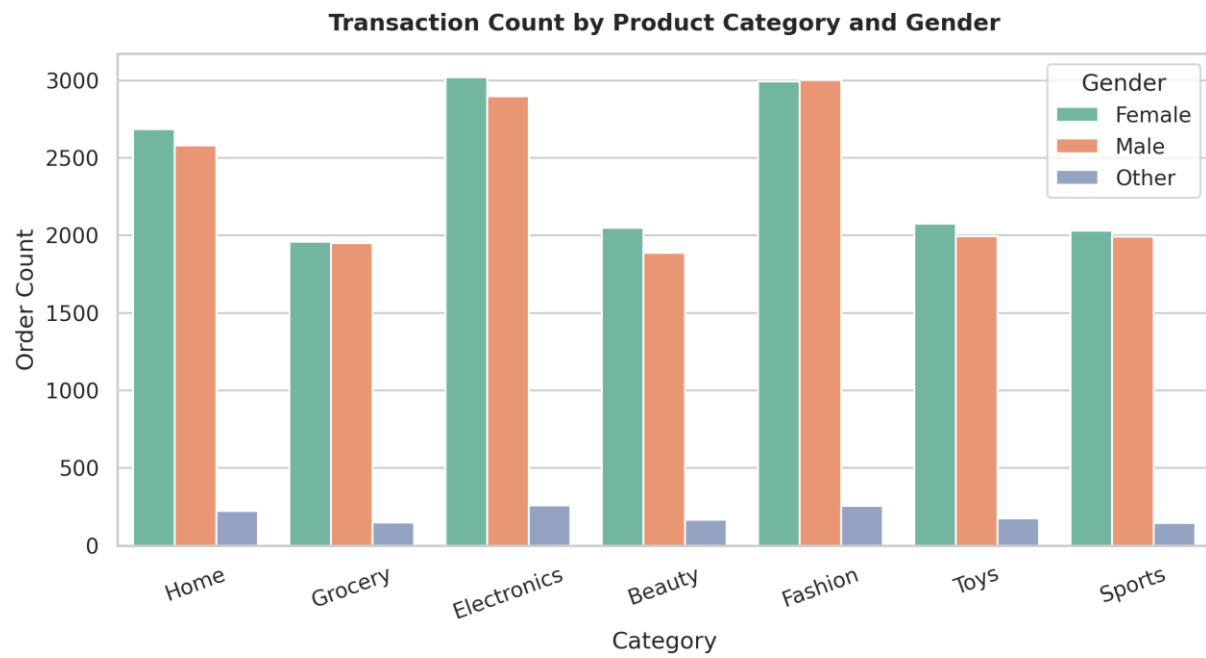
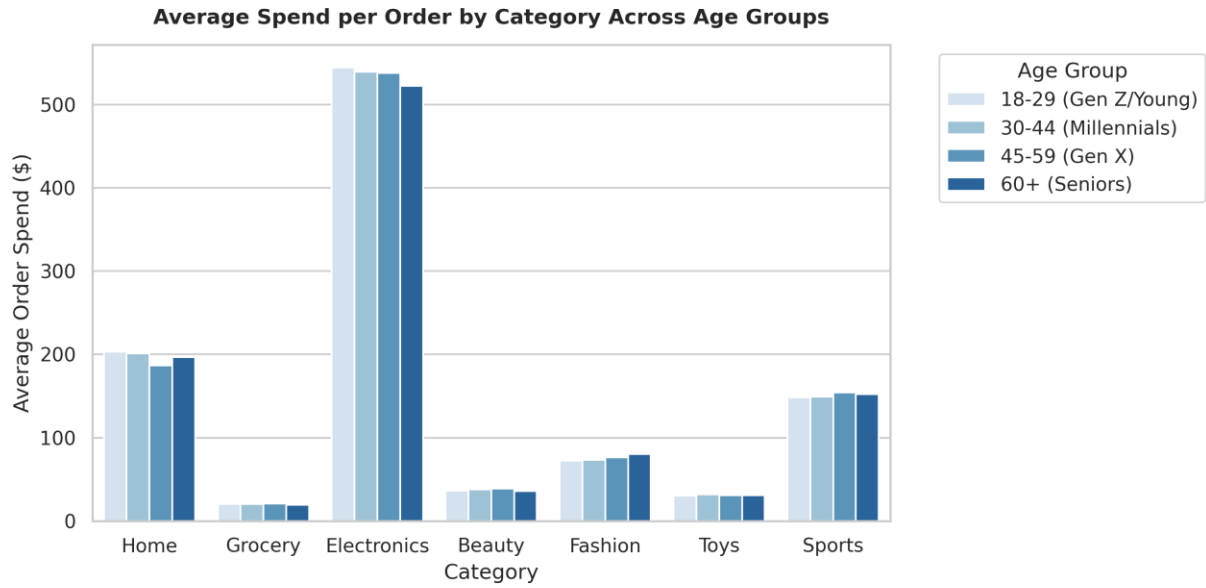


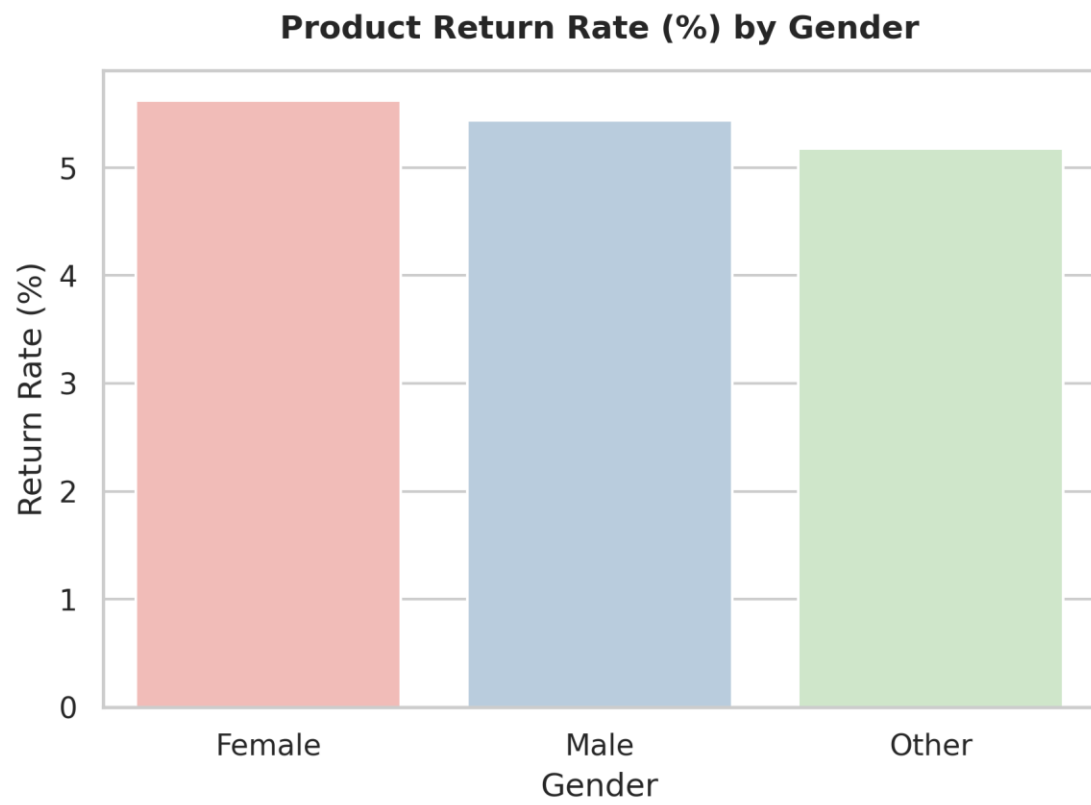
11. Average Order Value (AOV) Across Regions



12. Monthly Revenue Share Trend by Category Portfolio







BUSINESS INSIGHTS

High-Performing Categories

- Electronics drives revenue: Electronics made over \$3.4M in total sales. It is the main financial engine for the store.
- Home goods take second place: Home items brought in over \$1.1M in sales. They act as a steady secondary income source.

Low-Performing Categories

- Beauty and Grocery fall behind: Grocery brought in under \$100k, while Beauty stayed under \$150k.
- Toys struggle to sell: Toy sales stayed near the bottom of earnings, even with frequent price cuts.

Pricing & Revenue Dynamics

- Higher prices drive total spend: Unit price links strongly to total order spend ($r = 0.80$). Expensive items boost overall revenue far more than unit volume.
- Cart size has a smaller effect: Item quantity has only a modest link to total sales ($r = 0.31$). Buyers rarely add many items to a single order.
- Cross-selling needs work: Most orders hold just 1 to 3 items. The store relies on single pricey items instead of full carts.

Discount Policy Issues

- Discounts do not build cart size: Discount percentages show no connection to item quantities ($r = -0.01$). Price cuts fail to nudge buyers into adding extra products.
- Promotions hurt profit margins: Discount rates have zero link to total transaction value ($r = 0.00$). Site-wide discounts trim margins without growing sales.

Product Returns & Operations

- Fashion has the highest returns: Roughly 8.2% of fashion orders get returned. Poor sizing fits and photo mismatches likely cause most refunds.
- Electronics refunds cost money: Electronics has a 7.3% return rate. Returning high-ticket electronics hurts overall profit margins.
- Shipping speed does not cause returns: Returned orders take an average of 4 days to deliver, matching non-returned orders. Logistics delays do not trigger refunds.

Customer Demographics & Behavior

- Millennials spend the most: Customers aged 30–44 generate over 40% of revenue. They buy mostly Electronics and Home products.
- Seniors buy everyday goods: Shoppers aged 60+ spend less on Electronics, but they consistently buy Grocery and Beauty items.
- Gender splits show category trends: Men and women make up equal order counts (50% each), but women spend 12% more per cart in Fashion.
- Cards dominate checkout: Credit and debit cards handle over 60% of checkouts. Digital wallets take less than 10% of total payment volume.

BUSINESS IMPROVEMENT RECOMMENDATION

- Focus Procurement on High-Demand Categories: Reallocate 60% of inventory budget to high-revenue Electronics (\$3.4M+) and Home (\$1.1M+) targeting 30–44-years-old, while reducing warehouse space for stagnant Beauty and Grocery stock.
- Replace Flat Discounts with Spend Thresholds: Shift from site-wide price cuts (which show $r = 0.00$ correlation with total sales) to tiered offers like "Spend \$200 to get 15% off" to drive up cart quantity ($r = 0.31$).
- Reduce Return Leakage in Fashion & Electronics: Introduce interactive sizing guides for Fashion (8.2% return rate) and mandate pre-shipment checks for Electronics (7.3% return rate) to resolve product expectation gaps.
- Optimize Card Checkouts & Incentivize Digital Wallets: Streamline credit/debit card processing (which handles 60%+ of checkouts) while offering 5% cashback on digital wallets to raise adoption from under 10%.
- Bundle Slow-Moving SKUs with Bestsellers: Offer low-performing Beauty, Toy, or Grocery items as checkout add-ons with Electronics or Fashion orders to clear inventory and expand average cart sizes beyond 1 to 3 items.